AOSP 进阶丛书 · 第十三册 · ANDROID 13 (API 33)

SystemUI 通知体系

SystemUI 的身份（特权 App 而非系统服务）、CoreStartable 微服务模型、通知从 notify() 到状态栏的完整管线（NMS 排序/通道/速率/Bubbles）、最近任务截图与 Launcher3 状态驱动架构。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：用户看见的"系统" 2
第一章 SystemUI：不是服务，是特权应用 2
第二章 通知管线：notify() 到显示的完整链 3
第三章 最近任务与快照 5
第四章 Launcher3：桌面与图标 5
第五章 设计决策八问 6
附录 A 观测实操 6
附录 B 快问快答（14 条） 7
附录 C 自检题 7
第六章 启动链：SystemServer 到 CoreStartable 的微服务总线 7
第七章 一块全屏窗口：TYPE_NOTIFICATION_SHADE 的层级设计 11
第八章 通知进入 SystemUI：Listener→Coalescer→Collection→ListBuilder 14
第九章 通知行的诞生与"弹幕"判定：两段式 inflate 与 HUN 链 19
教科书习题集（分级） 26
术语表 29
排障手册：SystemUI 与通知问题定位 30
SystemUI 核心速查表 33

本册速览

SystemUI 是普通 App：SystemUIApplication:62 + CoreStartable 注册制
通知管线：notify()→NMS:349 排序/通道/速率→SystemUI 渲染
最近任务：TaskSnapshotController 截图+RecentTasks 重组
Launcher3：StatefulActivity 状态驱动 · LauncherModel 监听装/卸
设计决策：为什么 SystemUI 进程独立/通知为什么有通道/快照为什么在 system_server

前言：用户看见的"系统"

前十二册讲完了系统内部的机器；本册讲**用户看见的那个"系统"**：状态栏/通知/快捷设置/锁屏/最近任务——它们全部住在一个名叫 SystemUI 的**特权应用**里，加上作为桌面的 Launcher3，共同构成"用户体验的门面"。

体例不变：真实源码锚点 + 逐段注释 + 流程图 + 设计决策。本册主角：`frameworks/base/packages/SystemUI/`（SystemUI 主体）、`frameworks/base/services/core/.../notification/`（NMS）、`packages/apps/Launcher3/`（桌面）。

第一章 SystemUI：不是服务，是特权应用

设计洞察 · 学习目标

1. 说出 SystemUI 的进程名/uid/签名特征并解释"特权 App"含义；2. 复述 SystemServer 到 SystemUIService 的启动入口；3. 列出 CoreStartable 微服务模型的设计动机；4. 解释 SystemUI 崩溃后系统为何仍可存活并自动重启



图 1 SystemUI 的身份与出生（SystemUIApplication:62）、CoreStartable 微服务模型、主要组件清单与四条独立进程理由

1.1 身份逐段（真实锚点）

java frameworks/base/packages/SystemUI/…（骨架+讲解注释）

```
// SystemUIApplication.java:62
public class SystemUIApplication extends Application
implements Configuration.ProviderService {
// 讲解注释①：SystemUI 是一个普通 App——有 Application/有 Activity/有 Service
// 进程名 com.android.systemui，uid=SYSTEM（platform 签名特权）
// 讲解注释②：启动方式：SystemServer.java:3053 startSystemUi()
// → startService(SystemUIService) → Application.onCreate()
// → 依次启动各 CoreStartable（组件）
}
// CoreStartable.java:34
public abstract class CoreStartable implements Dumpable {
// 讲解注释③：SystemUI 内部的"微服务注册制"：
// StatusBar / Recents / QS / VolumeUI… 每个实现此类
// → Dependency.java 注入（ServiceLocator 模式）
// → 每个组件独立 onStart()，可 dumpsys 分组件查看
}
```

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释SystemUI 与通知的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

设计洞察 · 逐段精读：为什么 SystemUI 是独立 App 而非 system_server 线程

四条理由：**崩溃隔离**（UI 死了系统活着可自动重启）、**渲染独立**（UI 帧率不拖服务线程）、**独立 OTA**（Mainline 可单独升级 SystemUI 组件）、**内存配额独立**。面试高频"SystemUI 崩了会怎样"：init 检测 critical → 自动重启（闪一下状态栏恢复）。

第二章 通知管线：notify() 到显示的完整链

设计洞察 · 学习目标

1. 画出 notify() 从 App 通道到 SystemUI 渲染的四段链并标出 binder 跳点；2. 说出 NMS 入队的三道闸（通道/速率/桶）各自含义；3. 解释 NotificationSignalExtractor 链的排序机制；4. 用 dumpsys notification 定位"通知不显示"的五类原因



图 2 通知四段：App 通知通道→NMS 排序策略→SystemUI 渲染，含 Bubbles/速率/桶联动

2.1 NMS 核心逐段（真实锚点）

java NotificationManagerService.java（骨架+讲解注释）

```
// :349 public class NotificationManagerService extends SystemService
// ——SystemServer startOtherServices 启动（书2 §七）
//
// enqueueNotificationInternal(:6397/6404)
// 讲解注释①：权限/通道/速率三道闸：
// • 通道不存在 → dropped（8+ 必须先 createNotificationChannel）
// • 速率超限（mRateLimitExceeded）→ dropped
// • Standby RESTRICTED 桶（书9 §五）→ 静默
// 讲解注释②：NotificationRecord 入 mNotificationList（按排名排序）
// 排序由 NotificationSignalExtractor 链决定：
// Importance→Channel→ZenMode→Visibility→…各提取器逐级调整排名
// 讲解注释③：buzzBeepBlink——声音/振动/LED 三个通道策略在此
// 讲解注释④：notifyPostedListeners→SystemUI 的 NotificationListener
// binder 回调 → SystemUI 渲染面板
```

设计洞察 · 排障树："通知不显示"的排查序

通道开关→勿扰模式→importance 被降→速率限制→后台桶限制。dumpsys notification 一次看全这五项。面试高频："为什么同样代码有的设备响有的不响"——厂商策略差异+勿扰默认值差异。

第三章 最近任务与快照

设计洞察 · 学习目标

1. 描述 TaskSnapshotController 截图的时机与存储载体（GraphicBuffer）；2. 解释 RecentTasks 与 RecentsView 的分工；3. 追踪点击最近任务到 ATMS.startActivityFromRecents 的链路；4. 说明"秒开"假象由截图先行实现

TaskSnapshotController（wm/）在 Activity 切后台时截图（经 SurfaceFlinger 的 screenshot 接口→GraphicBuffer 存储）；RecentTasks（:109）管理任务栈重组（分屏/最近列表排序/重新打开）。**Launcher3 的 RecentsView 消费这两者**：截图显示+TaskKey 定位→点按→ATMS.startActivityFromRecents（书4 §一 的入口之一）。

`dumpsys activity recents` 看当前任务栈全表；`dumpsys window | grep snapshot` 看快照状态。快照的意义：**最近任务"秒开"的视觉假象**——先显示截图再让 App 追上真实帧。

第四章 Launcher3：桌面与图标

设计洞察 · 学习目标

1. 解释 StatefulActivity 状态驱动架构与常用状态；2. 说出 LauncherModel 监听装卸事件的系统接口（LauncherApps.Callback）；3. 描述 IconCache 的缓存层次与持久化；4. 说出预测栏与 UsageStats 的联动点

java packages/apps/Launcher3（骨架+讲解注释）

```
// Launcher.java:238
public class Launcher extends StatefulActivity<LauncherState>
// 讲解注释①：StatefulActivity=状态驱动 UI——NORMAL/APPLICATION/
// OVERVIEW 等状态各自定义可交互元素，切换即重绘
// LauncherModel.java:87
public class LauncherModel extends LauncherApps.Callback
implements InstallSessionTracker.Callback {
// 讲解注释②：模型层——监听 App 安装/卸载（LauncherApps 系统服务
// ↔ 书6 PMS 的安装事件），维护图标与 widget 布局数据库
// 讲解注释③：IconCache——App 图标/标签缓存（跨启动持久化）
}
```

Launcher3 的预测（13）：`packages/apps/Launcher3/src/.../predictor/` 基于使用统计的 App 预测排序（与书9 UsageStats 联动）。

第五章 设计决策八问

设计洞察 · 学习目标

1. 独立复述八问中任意四问的完整答案；2. 把每个答案对应到前四章的具体源码锚点；3. 识别每问背后的共性权衡（隔离/性能/可维护性）；4. 用自己的话向非系统背景同事解释"SystemUI 为什么是 App"

- **为什么 SystemUI 是 App 不是系统服务？** 崩溃隔离+渲染独立+独立 OTA+内存独立（第一章四条）。
- **为什么通知要走 NMS 而非直接给 SystemUI？** 排序/策略/速率/通道检查需要全局视角（权限数据在 system_server）——SystemUI 只是渲染者。
- **为什么 8+ 强制通知通道？** 用户需要细粒度控制（"关广告不关聊天"）；通道是"通知偏好组"的载体。
- **为什么最近任务用截图而非实时窗口？** 性能（截图是静态 GraphicBuffer）+隐私（后台 App 可能已变内容）；"秒开"是假象先给截图再让 App 追帧。
- **为什么 Launcher 用 StatefulActivity？** 桌面 UI 状态多（NORMAL/OVERVIEW/WIDGET_PICKER…）且切换频繁——状态驱动让每种状态的交互规则声明式定义。
- **为什么 LauncherModel 而非直接操作数据库？** 装卸事件异步到达（Binder 回调线程）→需统一调度主线程刷新→Model 是"事件→状态→UI"的中间层。
- **为什么通知排序是提取器链而非单一规则？** 排序因素多维（通道重要度×时效×用户偏好×勿扰）——链式提取器可按序调整且可插拔。
- **为什么 SystemUI 逐步 Dagger 化？** Dependency.get() 全局单例测试难；Dagger 编译期注入+可测试替换——工程可维护性的渐进改造。

附录 A 观测实操

目标	命令	看点
通知全表	dumpsys notification	排序/通道/速率/勿扰
状态栏	dumpsys statusbar	面板/通知行/可见性
最近任务	dumpsys activity recents	任务栈/快照
SystemUI 组件	dumpsys activity service SystemUI	CoreStartable 状态
Launcher	dumpsys activity launcher	图标/widget 布局

附录 B 快问快答（14 条）

- SystemUI 是系统服务吗？——不是，特权 App（uid SYSTEM、platform 签名）。
- SystemUI 崩了怎样？——init critical 自动重启。
- 通知不显示第一查？——通道开关（8+ createChannel 是前提）。
- 排序由谁决定？——NotificationSignalExtractor 链。
- Bubbles 是什么？——通知的浮动形态（对话类）。
- 通知速率限制在哪？——NMS enqueueNotificationInternal（:6397）。
- RESTRICTED 桶的通知？——被静默（书9 联动）。
- 最近任务截图谁截的？——TaskSnapshotController→SF screenshot。
- Launcher 什么架构？——StatefulActivity 状态驱动。
- 图标缓存在哪？——LauncherModel 的 IconCache（持久化）。
- Launcher3 怎么知道装了新 App？——LauncherApps.Callback（书6 PMS 事件）。
- 通知点击怎么打开 Activity？——书4 PendingIntent 代执行。
- SystemUI 可独立升级吗？——部分组件可（Mainline）。
- 学完本册？——书14 HAL 实战（从应用到硬件驱动的最后一跳）。

附录 C 自检题

- SystemUI 的四条独立进程理由。
- 通知五步链（通道→NMS 排序→速率→通知监听→渲染），标出 binder 跳点。
- 最近任务截图从截到显示的链路。
- 为什么通知通道是 8+ 硬要求？

第六章 启动链：SystemServer 到 CoreStartable 的微服务总线

> **设计洞察 · 学习目标**
>
> 1. 复述 SystemServer→SystemUIService→SystemUIApplication 三级跳的完整路径并标出跨进程点；2. 解释 Dagger @IntoMap/@ClassKey 多绑定如何取代配置数组；3. 说明 TreeMap 确定性排序与 SystemUIBootTiming trace 的工程价值；4. 用 dumpsys/perfetto 定位单个 CoreStartable 的启动耗时

这一段在整条链路中的位置：第一章回答了"SystemUI 是谁"（特权 App），本章回答"SystemUI 是如何被逐个组件组装起来的"。全链只有一次跨进程跳：`SystemServer.startSystemUi()`

里 `context.startServiceAsUser()`（书3 的 AMS.startService 路径），之后一切都在 `com.android.systemui` 进程内。链路为：SystemServer.startOtherServices（SystemServer.java:3053）→ startSystemUi（:3182）→ SystemUIService.onCreate（SystemUIService.java:66）→ SystemUIApplication.startServicesIfNeeded（SystemUIApplication.java:191）→ 逐个 `new` + `onStart()` 全部 CoreStartable。 StatusBar（即 CentralSurfaces，13 已更名）只是这条总线上的一个乘客——尽管它是最大的那个。



图 3 第六章补全第一章架构图的时序维度：二十余个 CoreStartable 按类名字典序依次 onStart

6.1 startServicesIfNeeded 逐段精读

真实代码走读（SystemUIApplication.java，已核实行号）。先看入口方法——排序与组装：

java frameworks/base/packages/SystemUI/src/com/android/systemui/SystemUIApplication.java:190-210（骨架+讲解注释）

```
public void startServicesIfNeeded() {
final String vendorComponent = SystemUIFactory.getInstance()
.getVendorComponent(getResources());
// 讲解注释①：TreeMap 按 Class.getName() 字典序排序——启动顺序不
// 依赖声明位置，任何两次开机、任何两台设备都得到完全一致的
// 初始化顺序。这是"可复现调试"的工程前提：崩溃堆栈里组件间
// 的先后关系永远可对比基线
Map<Class<?>, Provider<CoreStartable>> sortedStartables = new TreeMap<>(
Comparator.comparing(Class::getName));
sortedStartables.putAll(SystemUIFactory.getInstance().getStartableComponents());
sortedStartables.putAll(SystemUIFactory.getInstance().getStartableComponentsPerUser());
startServicesIfNeeded(
sortedStartables, "StartServices", vendorComponent);
```

```
}
```

再看真正干活的私有重载——串行启动、计时、补发开机回调：

java frameworks/base/packages/SystemUI/src/com/android/systemui/SystemUIApplication.java:220-281（骨架+讲解注释）

```
private void startServicesIfNeeded(
Map<Class<?>, Provider<CoreStartable>> startables,
String metricsPrefix,
String vendorComponent) {
if (mServicesStarted) { // 讲解注释②：幂等保护——
return; // SystemUIService 与其他入口可能
} // 重复触发，二次调用直接返回
mServices = new CoreStartable[startables.size() + (vendorComponent == null ? 0 : 1)];
if (!mBootCompleteCache.isBootComplete()) {
// 讲解注释③：开机竞态——启动晚于 BOOT_COMPLETED 广播就
// 收不到；直接读 sys.boot_completed 属性补齐
if ("1".equals(SystemProperties.get("sys.boot_completed"))) {
mBootCompleteCache.setBootComplete();
}
}
mDumpManager = mSysUIComponent.createDumpManager();
// 讲解注释④：boot timing trace——全程写入 SystemUIBootTiming
// 的 TimingsTraceLog，perfetto/systrace 按此名过滤
TimingsTraceLog log = new TimingsTraceLog("SystemUIBootTiming",
Trace.TRACE_TAG_APP);
log.traceBegin(metricsPrefix);
int i = 0;
for (Map.Entry<Class<?>, Provider<CoreStartable>> entry : startables.entrySet()) {
String clsName = entry.getKey().getName();
int j = i; // Copied to make lambda happy.
// 讲解注释⑤：逐组件计时——timeInitialization 内部对每个
// 组件再开一层 traceBegin/traceEnd，单组件卡顿一目了然
timeInitialization(
clsName,
() -> mServices[j] = startStartable(clsName, entry.getValue()),
log,
metricsPrefix);
i++;
}
if (vendorComponent != null) {
// 讲解注释⑥：厂商扩展位——config_systemUIVendorServiceComponent
// 指定的类（默认 VendorServices）最后启动，OEM 挂载私有组件
timeInitialization(
vendorComponent,
() -> mServices[mServices.length - 1] =
startAdditionalStartable(vendorComponent),
log,
metricsPrefix);
}
for (i = 0; i < mServices.length; i++) {
if (mBootCompleteCache.isBootComplete()) {
// 讲解注释⑦：若初始化期间开机已完成，立即补发
// onBootCompleted——保证"注册了就一定会收到"
mServices[i].onBootCompleted();
```

```
}
// 讲解注释⑧：每个 CoreStartable 注册进 DumpManager——此后
// dumpsys activity service SystemUI 按组件名分节输出
mDumpManager.registerDumpable(mServices[i].getClass().getName(), mServices[i]);
}
mSysUIComponent.getInitController().executePostInitTasks();
log.traceEnd();
mServicesStarted = true;
}
```

三个细节值得放大。**其一**，`startStartable` 走 `Provider.get()` 而非 `new`：Provider 是 Dagger 的惰性工厂，组件的真实构造被推迟到这一刻，构造函数里所有 `@Inject` 依赖在同一时间原子就绪——这就是第一章说的"微服务注册制"的兑现处。**其二**，`onBootCompleted` 的双保险（广播 + 属性补发）解决的是"监听者比广播来得晚"的经典竞态，与书7 中 PMS 启动期扫描的处理思路同源。**其三**，所有组件在**主线程串行**初始化：没有任何一个 CoreStartable 允许在构造函数里做耗时 IO，否则整条状态栏起不来。

6.2 从配置数组到 Dagger 多绑定

Android 9 及以前，AOSP 用 `res/values/config.xml` 里 `config_systemUIServiceComponents` 字符串数组列全类名，运行期反射构造。Android 13 只留下两条残余：`config_systemUIServiceComponentsPerUser`（仅 NotificationChannels 一条）与厂商扩展字符串。替代品是 Dagger 编译期多绑定——类名检查从"运行期 ClassNotFoundException"提前到"编译期报错"：

java frameworks/base/packages/SystemUI/src/com/android/systemui/dagger/SystemUICoreStartableModule.kt:53-115（骨架+讲解注释）

```
@Module
abstract class SystemUICoreStartableModule {
// 讲解注释①：@Binds + @IntoMap + @ClassKey 三连是 Dagger 的
// "多绑定"惯用法：把每个实现放进 Map<Class<*>, Provider<CoreStartable>>
// ——SystemUIApplication 里那个 sortedStartables 的内容
// 就是 Dagger 编译期生成的填充代码（DaggerSysUI.*_getStartables()）
/** Inject into AuthController. */
@Binds
@IntoMap
@ClassKey(AuthController::class)
abstract fun bindAuthController(service: AuthController): CoreStartable
/** Inject into KeyguardViewMediator. */
@Binds
@IntoMap
@ClassKey(KeyguardViewMediator::class)
abstract fun bindKeyguardViewMediator(sysui: KeyguardViewMediator): CoreStartable
/** Inject into NotificationChannels. */
@Binds
@IntoMap
@ClassKey(NotificationChannels::class)
// 讲解注释②：@PerUser 标记——该组件进入 PerUser 集合，只在
// 逻辑主用户进程里由 startSecondaryUserServicesIfNeeded 启动
@PerUser
abstract fun bindNotificationChannels(sysui: NotificationChannels): CoreStartable
```

```
// 讲解注释③：PowerUI/VolumeUI/StorageNotification/WMShell 等
// 二十余个组件同构绑定——"新增一个 SystemUI 组件"= 写一个
// CoreStartable 实现 + 加三行绑定，零反射零配置
}
```

java frameworks/base/services/java/com/android/server/SystemServer.java:3182-3190（骨架+讲解注释）

```
private static void startSystemUi(Context context, WindowManagerService windowManager) {
PackageManagerInternal pm = LocalServices.getService(PackageManagerInternal.class);
Intent intent = new Intent();
// 讲解注释①：组件名来自 pm.getSystemUiServiceComponent()——即
// config_systemUiServiceComponent 资源，默认指向 SystemUIService；
// OEM 换启动类只改资源不改 SystemServer
intent.setComponent(pm.getSystemUiServiceComponent());
intent.addFlags(Intent.FLAG_DEBUG_TRIAGED_MISSING);
// 讲解注释②：唯一的跨进程跳——AMS.startServiceAsUser 把启动
// 消息投给 com.android.systemui 进程（书3 binder 路径）
context.startServiceAsUser(intent, UserHandle.SYSTEM);
// 讲解注释③：WMS 同步获知"SystemUI 起了"——此刻起才允许
// 状态栏/导航栏等 systemui 窗口参与布局
windowManager.onSystemUiStarted();
}
```

设计洞察 · 设计洞察：为什么"确定性顺序"值得单独设计

TreeMap 字典序看似平淡，实为大型 UI 系统的运维基建：组件 A 依赖组件 B 的坑只有两种解法——显式依赖注入（本册所有组件的真依赖都走 Dagger 构造注入，与顺序无关）或显式 InitController.executePostInitTasks 的后置队列（SystemUIApplication:276），顺序本身永远只是观测变量而非控制变量。一旦有组件"碰巧依赖顺序"，这条链就不可维护了——面试可答："SystemUI 启动顺序由什么决定"= 类名字典序 + 依赖全部显式化。

交叉引用：组件清单与身份论证见第一章；启动完成的下一站——CentralSurfaces.start() 里拉起通知阴影窗口——见第七章；通知子系统的装配（NotificationsControllerImpl.initialize）见第八章。

第七章 一块全屏窗口：TYPE_NOTIFICATION_SHADE 的层级设计

设计洞察 · 学习目标

1. 说出通知阴影窗口的类型值、四个关键 FLAG 与布局Insets策略；2. 解释为什么锁屏/下拉阴影/通知列表共用一个窗口而非分窗口；3. 追踪 CentralSurfacesImpl.start() 到 WindowManager.addView 的建窗路径；4. 用 dumpsys window 验证该窗口的存在与参数

这一段在整条链路中的位置：第六章的总线把 CentralSurfaces（原 StatusBar）拉起来之后，它做的第一件"可见"的事就是**建窗口**——没有窗口，第二章讲的通知渲染管线就没有挂载点。链路：CentralSurfacesImpl.start()（CentralSurfacesImpl.java:912）→ mBarService.registerStatusBar(mCommandQueue)（:951，此处反向跨进程：SystemUI 把

CommandQueue 的 Binder 端点交给 system_server，书2 的通知操作下行通道由此打通）→ createAndAddWindows（:2431）→ NotificationShadeWindowControllerImpl.attach()（:2433）→ WindowManager.addView。注意一个容易误解的点：状态栏本体（TYPE_STATUS_BAR）与通知阴影（TYPE_NOTIFICATION_SHADE）是**两个窗口**，本章讲的是后者——那个真正承载通知行的全屏窗口。

7.1 attach() 逐段精读

真实代码走读（NotificationShadeWindowControllerImpl.java:215-252，已核实）：

java frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/phone/NotificationShadeWindowControllerImpl.java:215-252（骨架+讲解注释）

```
@Override
public void attach() {
// 讲解注释①：注释原话"the entire thing is made TRANSLUCENT"——
// 通知阴影包含下拉面板与半透明背景，因此整个窗口用
// TRANSLUCENT 像素格式并开启硬件加速
mLp = new LayoutParams(
ViewGroup.LayoutParams.MATCH_PARENT,
ViewGroup.LayoutParams.MATCH_PARENT,
LayoutParams.TYPE_NOTIFICATION_SHADE,
// 讲解注释②：四个 FLAG 的分工——NOT_FOCUSABLE（输入框
// 焦点让给 App，远程输入弹出时再动态改）、
// TOUCHABLE_WHEN_WAKING（抬手/息屏边缘仍可下拉）、
// SPLIT_TOUCH（分屏多点触控透传）、
// WATCH_OUTSIDE_TOUCH（点击阴影外区域收起面板）、
// DRAWS_SYSTEM_BAR_BACKGROUNDS（自绘状态栏底色）
LayoutParams.FLAG_NOT_FOCUSABLE
| LayoutParams.FLAG_TOUCHABLE_WHEN_WAKING
| LayoutParams.FLAG_SPLIT_TOUCH
| LayoutParams.FLAG_WATCH_OUTSIDE_TOUCH
| LayoutParams.FLAG_DRAWS_SYSTEM_BAR_BACKGROUNDS,
PixelFormat.TRANSLUCENT);
mLp.token = new Binder(); // 讲解注释③：窗口 token——WMS 用它
// 归组与鉴权，配额由 WindowToken 管理
mLp.gravity = Gravity.TOP;
// 讲解注释④：setFitInsetsTypes(0)——不避让任何系统 Insets!
// 状态栏/导航栏/刘海都在它"体内"，它必须覆盖整屏才有
// 下拉手势与挖孔区的完整控制权
mLp.setFitInsetsTypes(0 /* types */);
mLp.softInputMode = LayoutParams.SOFT_INPUT_ADJUST_RESIZE;
mLp.setTitle("NotificationShade"); // 讲解注释⑤：dumpsys window
// 里按这个名字定位本窗口
mLp.packageName = mContext.getPackageName();
// 讲解注释⑥：LAYOUT_IN_DISPLAY_CUTOUT_MODE_ALWAYS——内容延伸
// 进刘海区，状态栏图标行因此能画到挖孔两侧
mLp.layoutInDisplayCutoutMode = LAYOUT_IN_DISPLAY_CUTOUT_MODE_ALWAYS;
// 讲解注释⑦：BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE——从屏幕边缘
// 滑入即短暂显示系统栏；但注释说明 WMS 对该窗口禁用了这个
// 行为（由 SystemUI 自己控制显隐）
mLp.privateFlags |= PRIVATE_FLAG_BEHAVIOR_CONTROLLED;
mLp.insetsFlags.behavior = BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE;
```

```
// 讲解注释⑧：真正跨进程——Binder 调用进入 WMS，ViewRootImpl
// 建立，从此这个 View 树开始接收 Vsync（书4 窗口一章）
mWindowManager.addView(mNotificationShadeView, mLp);
mLpChanged.copyFrom(mLp); // 讲解注释⑨：mLpChanged 是后续动态
// 改属性的"影子副本"，每次改完
// 再统一 apply 到 WMS，避免频繁 IPC
onThemeChanged();
// 讲解注释⑩：与 KeyguardViewMediator#setupLocked 保持一致——
// 若此刻锁屏已在显示，立刻同步 setKeyguardShowing(true)，
// 否则会出现"通知阴影不知道锁屏在上面"的层叠错乱
if (mKeyguardViewMediator.isShowingAndNotOccluded()) {
setKeyguardShowing(true);
}
}
```

7.2 为什么锁屏、下拉、通知列表挤在同一个窗口

把 7.1 的注释串起来就能回答这个设计问题。**类型与层级**：`TYPE_NOTIFICATION_SHADE = FIRST_SYSTEM_WINDOW + 40`（frameworks/base/core/java/android/view/WindowManager.java:1590，即 z 序 2040），同族里 TYPE_STATUS_BAR 是 `FIRST_SYSTEM_WINDOW + 38`。也就是说阴影窗口恰好压在状态栏之上一层，而锁屏（Keyguard 由 KeyguardViewMediator 管理、其窗口类型更高）再压在阴影之上——**三明治结构**：下拉阴影 → 状态栏 → 锁屏，滑动解锁时由 LockscreenShadeTransitionController（statusbar/LockscreenShadeTransitionController.kt）在同一个窗口内做"锁屏内容下移、通知列表露出"的过渡。

组件	窗口类型	归属
状态栏图标行	TYPE_STATUS_BAR（系统层）	StatusBarWindowControllerImpl
通知阴影全屏窗	TYPE_NOTIFICATION_SHADE	本节 attach()
锁屏界面（Keyguard）	锁屏窗口（更高层）	KeyguardViewMediator（书2 侧管理）

为什么不拆成多个窗口？ 三个理由。第一，**触摸仲裁**：下拉手势要在"锁屏头像区"与"通知行"之间连续滑动，同一窗口内手势分发只需一层 View 逻辑，跨窗口则要做 WMS 级的触摸移交。第二，**Insets 的原子性**：`setFitInsetsTypes(0)` 让窗口无视全部系统 Insets，状态栏背景、导航栏遮挡、挖孔区自绘全部由 SystemUI 内部自己做——如果列表是一个独立窗口，这两套 Insets 策略就会在挖孔边缘出现一像素级接缝。第三，**背景模糊与遮挡关系**：阴影半透明背景要对 App 内容做 blur，WMS 模糊操作以窗口为单位，整屏单窗口让"模糊谁"变成一个简单问题。

设计洞察 · 设计洞察：一个 token 的窗口装下三种形态

attach() 里 `mLp.token = new Binder()` 声明了独立 token，这个窗口的显隐/参数全部走 mLpChanged 影子副本 + 单点 apply（本文件 setStatusbarExpanded/setQsExpansion 等十余个方法），这是"单一写者"模式：任何组件想改窗口参数都必须经过 NotificationShadeWindowController 接口集中仲裁，避免多模块并发改 LayoutParams 导致的状态抖动。面试高频："通知下拉动画为什么会卡"——先看该窗口是否被频繁 apply（dumpsys window 可见参数变更），再看 NotifPipeline 重建（见第八章）。

观测：`adb shell dumpsys window NotificationShade`（标题即 7.1 讲解注释⑤ 设置的 "NotificationShade"）可直接看到 mAttrs 全部参数与本节逐一对应。

交叉引用：该窗口内的通知列表由第八章的管线喂数据；窗口内 View 树的异步构建见第九章；状态栏窗口（TYPE_STATUS_BAR）与 CommandQueue 的注册次序见第六章与第二章。

第八章 通知进入 SystemUI：Listener→Coalescer→Collection→ListBuilder

设计洞察 · 学习目标

1. 追踪 onNotificationPosted 从 NMS 到 NotifCollection 的完整路径并标出线程切换点；2. 解释 NotifCollection 事件队列+整表重建的设计动机；3. 说明 onListenerConnected 的存量补发与 ranking 缺项兜底；4. 解释 RankingMap 500ms 合帧的节流策略

这一段在整条链路中的位置：第二章止步于 NMS 的 `mListeners.notifyPostedLocked`（NotificationManagerService.java:6292/:7473）——通知跨 Binder 落到 SystemUI 之后发生了什么，本章补全。管线在 13 里已经完成"新管线"改造，装配代码在 NotificationsControllerImpl.initialize（statusbar/notification/init/NotificationsControllerImpl.kt:93）：先 `notificationListener.registerAsSystemService()`（:100，把 SystemUI 注册成特权 NotificationListener，这一身份让它能看到全量通知——普通 App 的 Listener 只能看到自己被授权的），随后 :122 `newNotifPipelineInitializer.get().initialize(...)`，:191 `INITIALIZE_NEW_PIPELINE = true` 常量说明新管线恒装配（渲染是否走新管线另由 NotifPipelineFlags 开关控制）。装配顺序很讲究——NotifPipelineInitializer.initialize（collection/init/NotifPipelineInitializer.java:101-110）自上而下把六件套串成一条直线：

java frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/notification/collection/init/NotifPipelineInitializer.java:101-114（骨架+讲解注释）

```
// 讲解注释①：渲染层先挂——RenderStageManager 持有"列表就绪"
// 的回调端，等上游数据到达
mRenderStageManager.attach(mListBuilder);
// 讲解注释②：ListBuilder 逻辑组包层——分组/排序/过滤后的
// ListEntry 树在这里产出
mListBuilder.attach(mNotifCollection);
// 讲解注释③：Collection 数据层——NMS 事件在此变成
// NotificationEntry 集合（本章主角）
mNotifCollection.attach(mGroupCoalescer);
```

```
// 讲解注释④：Coalescer 合并层——挂在 NotificationListener
// 直下，做分组事件合批
mGroupCoalescer.attach(notificationService);
// 讲解注释⑤：注意 attach 方向与数据流相反——依赖注入时
// 上层把"接收事件的句柄"交给下层，数据从最底层
// NotificationListener 向上流到渲染
```

8.1 NotificationListener：入口的三件小事

真实代码走读（NotificationListener.java:102-217，已核实）：

java frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/NotificationListener.java:102-217（骨架+讲解注释）

```
@Override
public void onListenerConnected() {
// 讲解注释①：Listener 与 NMS 的 Binder 连接建立后的第一件事：
// 拉存量——SystemUI 启动晚于通知产生（第六章的开机竞态），
// 不拉存量通知面板就是空的
final StatusBarNotification[] notifications = getActiveNotifications();
if (notifications == null) {
Log.w(TAG, "onListenerConnected unable to get active notifications.");
return;
}
final RankingMap currentRanking = getCurrentRanking();
mMainExecutor.execute(() -> {
// 讲解注释②：binder 线程 → 主线程的唯一跳点（本类所有
// 回调都 mMainExecutor.execute）——下游管线可以放心
// 假设自己永远在主线程
// 讲解注释③：getActiveNotifications() 与 getCurrentRanking()
// 是两次独立 IPC，存在竞态：排名表可能缺个别通知的
// 条目。这里用 getRankingOrTemporaryStandIn 填充"临时
// 占位 Ranking"防下游崩溃（b/146011844）
final List<Ranking> newRankings = new ArrayList<>();
for (StatusBarNotification sbn : notifications) {
newRankings.add(getRankingOrTemporaryStandIn(currentRanking, sbn.getKey()));
}
final RankingMap completeMap = new RankingMap(newRankings.toArray(new Ranking[0]));
for (StatusBarNotification sbn : notifications) {
for (NotificationHandler listener : mNotificationHandlers) {
// 讲解注释④：存量通知以与新增完全相同的
// onNotificationPosted 语义回放——下游无需写
// "初始化"特判分支
listener.onNotificationPosted(sbn, completeMap);
}
}
for (NotificationHandler listener : mNotificationHandlers) {
listener.onNotificationsInitialized(); // 讲解注释⑤：存量回放完才宣布"初始化完成"
}
});
onSilentStatusBarIconsVisibilityChanged(
mNotificationManager.shouldHideSilentStatusBarIcons());
}
```

再看增量事件的入口与排名节流（NotificationListener.java:136-217）：

java frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/NotificationListener.java:136-217（骨架+讲解注释）

```
@Override
public void onNotificationPosted(final StatusBarNotification sbn,
final RankingMap rankingMap) {
if (DEBUG) Log.d(TAG, "onNotificationPosted: " + sbn);
if (sbn != null && !onPluginNotificationPosted(sbn, rankingMap)) {
mMainExecutor.execute(() -> {
// 讲解注释⑥：processForRemoteInput——处理远程输入
//  （通知内直接回复）所需的额外数据标记
processForRemoteInput(sbn.getNotification(), mContext);
for (NotificationHandler handler : mNotificationHandlers) {
handler.onNotificationPosted(sbn, rankingMap);
}
});
}
}
@Override
public void onNotificationRankingUpdate(final RankingMap rankingMap) {
if (DEBUG) Log.d(TAG, "onRankingUpdate");
if (rankingMap != null) {
// 讲解注释⑦：排名更新先进队列再统一派发——自己维护
// Deque 保证事件相对顺序（remove/add 与 ranking 的
// 一致性），注释原话：RankingMap 必须与当前通知集精确一致
RankingMap r = onPluginRankingUpdate(rankingMap);
mRankingMapQueue.addLast(r);
mMainExecutor.execute(mDispatchRankingUpdateRunnable);
}
}
private void dispatchRankingUpdate() {
RankingMap r = mRankingMapQueue.pollFirst();
if (r == null) {
Log.wtf(TAG, "mRankingMapQueue was empty!");
}
if (!mRankingMapQueue.isEmpty()) {
// 讲解注释⑧：500ms 合帧——高频排名更新（例如音量条
// 连续刷 progress）时，只有队列排空或距首跳过超过
// MAX_RANKING_DELAY_MILLIS(:58) 才真正派发一次
final long now = mSystemClock.elapsedRealtime();
if (mSkippingRankingUpdatesSince == -1) {
mSkippingRankingUpdatesSince = now;
}
final long timeSkippingRankingUpdates = now - mSkippingRankingUpdatesSince;
if (timeSkippingRankingUpdates < MAX_RANKING_DELAY_MILLIS) {
return; // 讲解注释⑨：跳过本次，等队列排空——
// 通知排序的"防抖"就在这五行里
}
}
mSkippingRankingUpdatesSince = -1;
for (NotificationHandler handler : mNotificationHandlers) {
handler.onNotificationRankingUpdate(r);
}
}
```

8.2 NotifCollection：事件账本与整表重建

下游 NotifCollection（collection/NotifCollection.java:140）不再"来一条改一条"，而是把每个事件记账进 mEventQueue，然后**整表重建**。先看新增/更新路径的真实代码（NotifCollection.java:393-487 节选，已核实）：

java frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/notification/collection/NotifCollection.java:393-487（骨架+讲解注释）

```
private void onNotificationPosted(StatusBarNotification sbn, RankingMap rankingMap) {
// 讲解注释①：主线程断言——管线的一切状态变化都发生在
// 主线程，锁的复杂度被整体消灭（书5 线程模型的对照面）
Assert.isMainThread();
postNotification(sbn, requireRanking(rankingMap, sbn.getKey()));
applyRanking(rankingMap);
// 讲解注释②：记账完统一触发重建——多个事件在同一帧到达时
// 只做一次重建（合并语义在 dispatchEventsAndRebuildList）
dispatchEventsAndRebuildList();
}
private void postNotification(
StatusBarNotification sbn,
Ranking ranking) {
NotificationEntry entry = mNotificationSet.get(sbn.getKey());
if (entry == null) {
// A new notification!
// 讲解注释③：key = 包名+用户+tag+id——同一通知的更新
// 会命中这里之外的 else 分支
entry = new NotificationEntry(sbn, ranking, mClock.uptimeMillis());
mEventQueue.add(new InitEntryEvent(entry));
mEventQueue.add(new BindEntryEvent(entry, sbn));
mNotificationSet.put(sbn.getKey(), entry);
mLogger.logNotifPosted(sbn.getKey());
// 讲解注释④：EntryAddedEvent 入队——所有
// NotifCollectionListener 的 onEntryAdded 都由后续
// dispatch 统一触发（HUN 判定的入口，见第九章）
mEventQueue.add(new EntryAddedEvent(entry));
} else {
// Update to an existing entry
// 讲解注释⑤：更新路径——通知"再活过来"：先撤销本地
// dismiss/生命周期延长/拦截三种暂挂状态（见下文删除
// 路径），mCancellationReason 归零，再换上新 sbn。
// 不换 NotificationEntry 对象——View 与监听器天然保住
// 引用（第九章的"复用 row"由此而来）
cancelLocalDismissal(entry);
cancelLifetimeExtension(entry);
cancelDismissInterception(entry);
entry.mCancellationReason = REASON_NOT_CANCELED;
entry.setSbn(sbn);
mEventQueue.add(new BindEntryEvent(entry, sbn));
mLogger.logNotifUpdated(sbn.getKey());
mEventQueue.add(new EntryUpdatedEvent(entry, true /* fromSystem */));
}
}
```

删除路径是"记账→可否决"的三步式（NotifCollection.java:412-433）：

java frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/notification/collection/NotifCollection.java:412-433（骨架+讲解注释）

```
private void onNotificationRemoved(
StatusBarNotification sbn,
RankingMap rankingMap,
int reason) {
Assert.isMainThread();
mLogger.logNotifRemoved(sbn.getKey(), reason);
final NotificationEntry entry = mNotificationSet.get(sbn.getKey());
if (entry == null) {
// 讲解注释⑥：删除到达但集合里没有——例如 lifetime 被
// 延长期间 NMS 已取消；TODO 注释预告未来会抛异常，
// 现在只记日志
mLogger.logNoNotificationToRemoveWithKey(sbn.getKey(), reason);
return;
}
// 讲解注释⑦：先记账"取消原因"，再走 tryRemoveNotification——
// 生命周期扩展器（NotifLifetimeExtender，如屏幕隐藏通知
// 要等动画结束）可否决删除
entry.mCancellationReason = reason;
tryRemoveNotification(entry);
applyRanking(rankingMap);
dispatchEventsAndRebuildList();
}
```

设计洞察 · 设计洞察：为什么"整表重建"优于"增量修补"

NotifCollection 的每条事件都进 mEventQueue，重建（ShadeListBuilder.buildList，collection/ShadeListBuilder.java:384）每次都从 mNotificationSet 全量重新过滤/分组/排序产出不可变的新列表——UI 永远只消费"某时刻的快照"。代价是重建成本，收益是三个：状态不可能进入中间态（无并发修改）、任何插件/协调器（NotifCoordinators，如气泡）对列表的改写都自动参与下一次重建、调试时 ListDumper 可打印完整决策链。面试高频："为什么 Android 13 重写通知管线"——增量路径的 group 嵌套与动画状态组合爆炸，快照+重建把状态机从 N 变量收敛为单输入单输出。



图 4 第八章把第二章四段图的第三段（SystemUI 渲染）展开为六层：Listener→Coalescer→Collection→ListBuilder→RenderStage→View

观测：`adb shell dumpsys activity service SystemUI` 输出里 NotifPipeline 与 NotifCollection 分节可见 mNotificationSet 全表与事件日志（8.1/8.2 的组件都在第六章⑧注释处注册了 DumpManager）。

交叉引用：上游 NMS 的三道闸与排序链见第二章；列表如何变成 View 树见第九章；书9 的 App Standby 桶在 NMS 侧就已决定通知"能不能来"，本章管不到被拦下的那些。

第九章 通知行的诞生与"弹幕"判定：两段式 inflate 与 HUN 链

设计洞察 · 学习目标

1. 复述通知行 View 的两段式异步构建（row 骨架→content 内容）及其线程；2. 解释 RowInflaterTask 里 Throwable 起源栈的调试价值；3. 完整走读 HUN 判定链：HeadsUpController→InterruptStateProvider→HeadsUpManager；4. 说出 HUN 的最短显示时间与 pin 条件

这一段在整条链路中的位置：第八章结束时，新管线产出了一份"应该显示哪些通知"的不可变列表，但列表里每个 NotificationEntry 还没有 View——渲染层（RenderStageManager）随即调 NotificationRowBinderImpl.inflateViews 把数据变成 ExpandableNotificationRow。与此同时，一份高优先级通知要不要"横幅弹出"（heads-up，俗称弹幕/悬浮通知）是**另一条并行链**：HeadsUpController 监听 NotifCollectionListener.onEntryAdded，走完整判定链后才把 entry 交给

HeadsUpManager。两条链共享第八章的同一批事件，这正是新管线"事件驱动、多订阅者"的形态。

9.1 两段式异步 inflate

第一段：row 骨架。RowInflaterTask 借 androidx 的 AsyncLayoutInflater 在后台线程解 XML，主线程只接成品（row/RowInflaterTask.java:35-88 全文走读，已核实）：

java frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/notification/row/RowInflaterTask.java:35-88（骨架+讲解注释）

```
public class RowInflaterTask implements InflationTask,
AsyncLayoutInflater.OnInflateFinishedListener
{
private static final String TAG = "RowInflaterTask";
private static final boolean TRACE_ORIGIN = true;
private RowInflationFinishedListener mListener;
private NotificationEntry mEntry;
private boolean mCancelled;
private Throwable mInflateOrigin; // 讲解注释①：起源栈——见 inflate()
@Inject
public RowInflaterTask() {
}
public void inflate(Context context, ViewGroup parent, NotificationEntry entry,
RowInflationFinishedListener listener) {
if (TRACE_ORIGIN) {
// 讲解注释②：new Throwable() 只是抓栈不抛出——异步任务
// 失败时 onInflateFinished 已不在调用者的堆栈里，没有
// 这份快照就永远不知道"这行是谁要求 inflate 的"
mInflateOrigin = new Throwable("inflate requested here");
}
mListener = listener;
AsyncLayoutInflater inflater = new AsyncLayoutInflater(context);
mEntry = entry;
// 讲解注释③：把本任务挂到 entry 上——第八章的通知更新若
// 在 inflate 完成前又到达，调用方靠 entry.getInflationTask()
// 能 abort 掉过期的任务（InflationTask 接口的用途）
entry.setInflationTask(this);
// 讲解注释④：AsyncLayoutInflater 在后台线程解析
// status_bar_notification_row 布局，主线程只回调解成品——
// 一次 XML+反射的十几毫秒不占主线程
inflater.inflate(R.layout.status_bar_notification_row, parent, this);
}
@Override
public void abort() {
mCancelled = true; // 讲解注释⑤：置位即可——回调里判空丢弃
}
@Override
public void onInflateFinished(View view, int resid, ViewGroup parent) {
if (!mCancelled) {
try {
mEntry.onInflationTaskFinished();
// 讲解注释⑥：回调第二段（NotificationRowBinderImpl 的
// lambda）——继续绑定控制器与内容视图
mListener.onInflationFinished((ExpandableNotificationRow) view);
```

```
} catch (Throwable t) {
if (mInflateOrigin != null) {
// 讲解注释⑦：把"谁发起的"作为 suppressed 异常
// 一起打出来——一行日志同时看到崩溃点与发起点
Log.e(TAG, "Error in inflation finished listener: " + t, mInflateOrigin);
t.addSuppressed(mInflateOrigin);
}
throw t;
}
}
}
public interface RowInflationFinishedListener {
void onInflationFinished(ExpandableNotificationRow row);
}
}
```

第二段：控制器与内容。骨架到手后 NotificationRowBinderImpl 接管——挂控制器、绑图标、压内容视图（collection/inflation/NotificationRowBinderImpl.java:124-162，已核实）：

java frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/notification/collection/inflation/NotificationRowBinderImpl.java:124-162（骨架+讲解注释）

```
@Override
public void inflateViews(
NotificationEntry entry,
NotifInflater.Params params,
NotificationRowContentBinder.InflationCallback callback)
throws InflationException {
if (params == null) {
// 讲解注释①：旧管线可不传 params——断言只在两种管线
// 切换期有意义（NotifPipelineFlags 开关）
mNotifPipelineFlags.checkLegacyPipelineEnabled();
}
ViewGroup parent = mListContainer.getViewParentForNotification(entry);
if (entry.rowExists()) {
// 讲解注释②：复用路径——通知更新时 row 已存在：只刷
// 图标+reset 状态+重绑内容，不重新解 XML（与第八章
// "不换 entry 对象"一脉相承）
mIconManager.updateIcons(entry);
ExpandableNotificationRow row = entry.getRow();
row.reset();
updateRow(entry, row);
inflateContentViews(entry, params, row, callback);
} else {
// 讲解注释③：首建路径——图标在异步线程生成（大图标/
// 小图标两级缓存），row 骨架交给 9.1 的 RowInflaterTask
mIconManager.createIcons(entry);
mRowInflaterTaskProvider.get().inflate(mContext, parent, entry,
row -> {
// 讲解注释④：每个 row 配一个 Dagger 子组件
//  （ExpandableNotificationRowComponent）——
// 控制器 RowController 的依赖按行隔离，
// 泄漏半径=单行
ExpandableNotificationRowComponent component =
mExpandableNotificationRowComponentBuilder
.expandableNotificationRow(row)
```

```
.notificationEntry(entry)
.onExpandClickListener(mPresenter)
.listContainer(mListContainer)
.build();
ExpandableNotificationRowController rowController =
component.getExpandableNotificationRowController();
rowController.init(entry);
entry.setRowController(rowController);
bindRow(entry, row);
updateRow(entry, row);
// 讲解注释⑤：第三小段——内容视图（大头/摘要/
// 自定义 RemoteViews）再走 RowContentBindStage
// 的异步 rebind，完成后回调 callback
inflateContentViews(entry, params, row, callback);
});
}
}
```

所以完整地说，一条通知行的诞生是**三小段**：row 骨架（后台线程解 XML）→ 控制器绑定（主线程）→ 内容视图（RowContentBindStage 异步 rebind，见 inflateContentViews 内 `mRowContentBindStage.requestRebind`，NotificationRowBinderImpl.java:262）。三段各自可取消、可合并，用户下拉到一半的通知取消后不再浪费任何一段。

9.2 HUN 判定链：从事件到横幅

第八章的 `onEntryAdded` 事件有一个特殊订阅者。真实代码走读（interruption/HeadsUpController.java:84-133，已核实）：

java frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/notification/interruption/HeadsUpController.java:84-133（骨架+讲解注释）

```
private NotifCollectionListener mCollectionListener = new NotifCollectionListener() {
@Override
public void onEntryAdded(NotificationEntry entry) {
if (mInterruptStateProvider.shouldHeadsUp(entry)) {
// 讲解注释①：先判定再绑定视图——判定链（9.3）说"否"
// 就什么都不发生；说"是"才异步把 HUN 视图绑出来
mHeadsUpViewBinder.bindHeadsUpView(
entry, HeadsUpController.this::showAlertingView);
}
}
@Override
public void onEntryUpdated(NotificationEntry entry) {
// 讲解注释②：更新要重新仲裁——重要性被降级/勿扰开启的
// 通知在这里被"请下屏幕"
updateHunState(entry);
}
@Override
public void onEntryRemoved(NotificationEntry entry, int reason) {
stopAlerting(entry); // 讲解注释③：删通知先撤横幅
}
@Override
public void onEntryCleanUp(NotificationEntry entry) {
```

```
// 讲解注释④：pipeline 清理 entry 时中止未完成的绑定——
// 回调悬空导致的"幽灵横幅"被这行堵死
mHeadsUpViewBinder.abortBindCallback(entry);
}
};
private void showAlertingView(NotificationEntry entry) {
// 讲解注释⑤：真正的入列——HeadsUpManager（AlertingNotificationManager
// 子类）开始计时管理横幅的生老病死
mHeadsUpManager.showNotification(entry);
if (!mStatusBarStateController.isDozing()) {
// 讲解注释⑥：亮屏时立刻标记"已看"——静默图标计数、
// LED 不再闪，都依赖这次回写 NMS
setNotificationShown(entry.getSbn());
}
}
private void updateHunState(NotificationEntry entry) {
boolean hunAgain = alertAgain(entry, entry.getSbn().getNotification());
boolean shouldHeadsUp = mInterruptStateProvider.shouldHeadsUp(entry);
final boolean wasHeadsUp = mHeadsUpManager.isAlerting(entry.getKey());
if (wasHeadsUp) {
if (shouldHeadsUp) {
mHeadsUpManager.updateNotification(entry.getKey(), hunAgain);
} else {
// 讲解注释⑦：不再有资格——立刻移除（false=非强制，
// 尊重最短显示时间，见 9.3）
mHeadsUpManager.removeNotification(entry.getKey(), false);
}
} else if (shouldHeadsUp && hunAgain) {
mHeadsUpViewBinder.bindHeadsUpView(entry, mHeadsUpManager::showNotification);
}
}
```

判定本体在 NotificationInterruptStateProviderImpl（interruption/NotificationInterruptStateProviderImpl.java:166-238，已核实）。注意入口先分屏态：**dozing（息屏/AOD）走 pulse 判定，awake 走 HUN 判定**，两套标准：

java frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/notification/interruption/NotificationInterruptStateProviderImpl.java:166-238（骨架+讲解注释）

```
@Override
public boolean shouldHeadsUp(NotificationEntry entry) {
if (mStatusBarStateController.isDozing()) {
return shouldHeadsUpWhenDozing(entry); // 讲解注释①：AOD 呼吸判定
} else {
return shouldHeadsUpWhenAwake(entry); // 讲解注释②：横幅判定
}
}
private boolean shouldHeadsUpWhenAwake(NotificationEntry entry) {
StatusBarNotification sbn = entry.getSbn();
if (!mUseHeadsUp) {
mLogger.logNoHeadsUpFeatureDisabled();
return false; // 讲解注释③：总开关（开发者选项可关）
}
if (!canAlertCommon(entry)) {
return false; // 讲解注释④：通用否决——见下
```

```
}
if (!canAlertAwakeCommon(entry)) {
return false; // 讲解注释⑤：亮屏专属否决（如正在通话）
}
if (isSnoozedPackage(sbn)) {
mLogger.logNoHeadsUpPackageSnoozed(sbn);
return false; // 讲解注释⑥：用户上滑"该应用全部横幅"后的包级 snooze
}
boolean inShade = mStatusBarStateController.getState() == SHADE;
if (entry.isBubble() && inShade) {
mLogger.logNoHeadsUpAlreadyBubbled(sbn);
return false; // 讲解注释⑦：气泡已展开时不再横幅——两种打扰形态互斥
}
if (entry.shouldSuppressPeek()) {
mLogger.logNoHeadsUpSuppressedByDnd(sbn);
return false; // 讲解注释⑧：勿扰（NMS 排名时写进 entry，第二章提取器产出）
}
if (entry.getImportance() < NotificationManager.IMPORTANCE_HIGH) {
mLogger.logNoHeadsUpNotImportant(sbn);
return false; // 讲解注释⑨：核心闸门——只有 HIGH/MAX（渠道配了声音/横幅）才有资格
}
boolean isDreaming = false;
try {
isDreaming = mDreamManager.isDreaming();
} catch (RemoteException e) {
Log.e(TAG, "Failed to query dream manager.", e);
}
boolean inUse = mPowerManager.isScreenOn() && !isDreaming;
if (!inUse) {
mLogger.logNoHeadsUpNotInUse(sbn);
return false; // 讲解注释⑩：屏幕没亮或在做梦（屏保）不弹
}
for (int i = 0; i < mSuppressors.size(); i++) {
if (mSuppressors.get(i).suppressAwakeHeadsUp(entry)) {
mLogger.logNoHeadsUpSuppressedBy(sbn, mSuppressors.get(i));
return false; // 讲解注释⑪：策略插件最后一票
}
}
mLogger.logHeadsUp(sbn);
return true;
}
```

`canAlertCommon`（同文件 :291 起）是两种形态共用的否决集：分组抑制告警（`suppressAlertingDueToGrouping`，组内只许 summary 响）、抑制器全局否决、刚弹过全屏意图（防"全屏来电+横幅"双份打扰）、锁屏可见性规则（KeyguardNotificationVisibilityProvider）。**判定链共十余关卡，但全部是"即时、无副作用"的谓词**——真正的副作用（计时、pin、响铃回写）只在 HeadsUpManager.showNotification（AlertingNotificationManager.java:68）里发生。

9.3 显示时长与 pin：横幅的生老病死

进入 HeadsUpManager 后，横幅由两条规则管理生死：**最短显示时间**（不许早退）与**自动消失计时**（不许久留），外加一个状态相关的 **pin**（钉在屏幕顶端）。真实代码走读（

AlertingNotificationManager.java:318-334 与 HeadsUpManagerPhone.java:366-372，已核实）：

java frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/AlertingNotificationManager.java:318-334 与 phone/HeadsUpManagerPhone.java:366-372（骨架+讲解注释）

```
public void updateEntry(boolean updatePostTime) {
long currentTime = mClock.currentTimeMillis();
// 讲解注释①：最短显示时间——无论多快被更新/降级，
// 横幅至少存活到 mEarliestRemovaltime，防止"闪一下"
mEarliestRemovaltime = currentTime + mMinimumDisplayTime;
if (updatePostTime) {
// 讲解注释②：hunAgain=true（通知又该响）时刷新
// 出现时刻——横幅"续命"的数据基础
mPostTime = Math.max(mPostTime, currentTime);
}
removeAutoRemovalCallbacks();
if (!isSticky()) {
// 讲解注释③：自动消失计时——finishTime 由
// mAutoDismissNotificationDecay（R.integer.heads_up_notification_decay 基准值，
// 实际叠加用户横幅时长偏好，见 policy/HeadsUpManager.java:465）
// + postTime 推算，与最短时间取 max
long finishTime = calculateFinishTime();
long removeDelay = Math.max(finishTime - currentTime, mMinimumDisplayTime);
// 讲解注释④：主线程 Handler.postDelayed 到点自动撤——
// 撤之前用户点开/上滑会走 removeNotification 抢先清理
mHandler.postDelayed(mRemoveAlertRunnable, removeDelay);
}
}
// 讲解注释⑤：isSticky 默认 false；子类可让横幅"粘屏"，
// 只能由外部显式移除（如来电类横幅的厂商定制）
@Override
protected boolean shouldHeadsUpBecomePinned(NotificationEntry entry) {
// 讲解注释⑥：pin 条件——普通锁屏态（SHADE 且面板未展开）
// 才把横幅"钉"在屏幕顶端，展开阴影时不再钉
boolean pin = mStatusBarState == StatusBarState.SHADE && !mIsExpanded;
if (mBypassController.getBypassEnabled()) {
// 讲解注释⑦：指纹/人脸绕过锁屏（bypass）模式下，锁屏态也钉
pin |= mStatusBarState == StatusBarState.KEYGUARD;
}
return pin || super.shouldHeadsUpBecomePinned(entry);
}
```

pin 与自动消失是独立的：pin 的横幅到点后先"缩回状态栏后方"（HeadsUpStatusBarView 的过渡），不 pin 的横幅到点直接移出；用户展开阴影时 pinned 行会被收纳进列表。这条状态线与第八章的 entry/row 复用共同保证了"横幅消失、列表行还在"的连续体验。

> **设计洞察 · 设计洞察：判定与执行分离的中枢价值**
> shouldHeadsUp 是纯函数（entry 进，boolean 出），HeadsUpManager 是状态机（计时/延寿/pin/snooze）。分离后：更新事件可直接复用判定重仲裁（9.2 注释⑦），AOD 与亮屏两形态共享同一份通用否决集，测试无需 mock 计时器。面试高频："为什么横幅会自动消失又弹回来"——最短显示时间（mMinimumDisplayTime，policy/HeadsUpManager.java:88 读 R.integer.heads_up_notification_minimum_time）保护"不早退"，自动消失时长基于 mAutoDismissNotificationDecay（:89）叠加用户偏好与扩展时长（HeadsUpManagerPhone.java:502 的 mExtensionTime），更新事件 hunAgain=true 会重置计时（AlertingNotificationManager.updateEntry，:318-334）。

观测：`adb shell dumpsys activity service SystemUI | grep -A8 HeadsUp`（或 `dumpsys notification --noredact | grep -i "hun\|heads"`）可看当前横幅集合与 snooze 列表；打不开横幅先查 9.2 注释⑨ 的重要度门槛（第二章 buzzBeepBlink 的同源数据）。

交叉引用：NMS 侧"响不响"的判定见第二章 buzzBeepBlink；横幅视图挂载在第七章的全屏窗口内；本链入口（onEntryAdded）来自第八章事件队列；气泡互斥（注释⑦）与第五章"Bubbles 是通知的浮动形态"呼应。

教科书习题集（分级）

第一级：简答题（检验概念理解）

- **1. SystemUI 是系统服务吗？给出至少三层证据。**

参考答案要点：不是。证据：独立进程 com.android.systemui（不是 system_server 线程）；uid 为 SYSTEM 且 platform 签名（特权 App）；由 SystemServer.startSystemUi（SystemServer.java:3182）以 startServiceAsUser 拉起——启动方式也是 App 的 Service。

- **2. SystemUI 崩溃后屏幕上会发生什么？谁负责把它救回来？**

参考答案要点：状态栏/通知/锁屏 UI 整体消失一瞬，系统其余部分（书2 的 system_server）完全不受影响；init 检测到 critical 进程退出自动重启，SystemUIApplication 重新走第六章启动链。

- **3. 为什么 Android 8.0 起发布通知必须先 createNotificationChannel？**

参考答案要点：通道是"通知偏好组"的载体，用户要"关广告不关聊天"的细粒度控制；通道不存在时 NMS 在 enqueueNotificationInternal 三道闸处直接丢弃，App 收不到回执但通知就是不来。

- **4. NMS 入队的三道闸分别是什么？**

参考答案要点：通道检查（不存在/关闭→dropped）、速率限制（mRateLimitExceeded，NMS:6397 起）、App Standby 桶（书9 RESTRICTED 静默）；三道闸都过才入 mNotificationList 并 notifyPostedListeners。

- **5. 通知排序由什么机制决定？举两个提取器例子。**

参考答案要点：NotificationSignalExtractor 链逐级调整 NotificationRecord 的排名分，可插拔；例子：ImportanceExtractor（渠道重要度）、ZenModeExtractor（勿扰降权），还有 NotificationGroupExtractor 等按配置顺序串行执行。

- **6. CoreStartable 是什么？SystemUIApplication 如何决定它们的启动顺序？**

参考答案要点：SystemUI 内部的"微服务"接口（onStart/onBootCompleted/dump）；实现类经 Dagger @IntoMap+@ClassKey 收进 Map，SystemUIApplication.java:197 用 TreeMap 按类名字典序排序后串行 onStart——顺序确定性是显式设计。

- **7. TYPE_NOTIFICATION_SHADE 窗口的 setFitInsetsTypes 传了什么？为什么？**

参考答案要点：传 0（不避让任何系统 Insets）；该窗口要覆盖整屏才能拿到下拉手势、自绘状态栏底色与挖孔区（CUTOUT_MODE_ALWAYS），状态栏/导航栏/刘海都在它"体内"。

- **8. 通知从 NMS 到达 SystemUI 后先落在哪个组件？回调最终在哪个线程执行？**

参考答案要点：NotificationListener（NotificationListener.java:137 onNotificationPosted）；binder 线程收到后一律 mMainExecutor.execute 切主线程，下游 NotifCollection 可用 Assert.isMainThread 免锁。

- **9. 一条通知行的 View 构建分几段？各段在什么线程？**

参考答案要点：三小段：RowInflaterTask 用 AsyncLayoutInflater 后台线程解 row 骨架 XML→主线程挂 ExpandableNotificationRowController 与图标→RowContentBindStage 异步 rebind 内容视图（RemoteViews），全部可取消可合并。

- **10. 渠道重要度至少要到几级才有横幅（HUN）资格？在哪里判的？**

参考答案要点：IMPORTANCE_HIGH；NotificationInterruptStateProviderImpl.shouldHeadsUpWhenAwake（:185 起）中 `entry.getImportance() < IMPORTANCE_HIGH` 直接 false，与第二章 buzzBeepBlink 的服务端响铃判定同源。

第二级：分析题（检验机制理解）

- **11. 完整追踪"App 调 notify() 到横幅弹出"的调用链，标注每次线程切换与 binder 跳点。**

参考答案要点：App notify→binder→NMS enqueueNotificationInternal（三道闸→SignalExtractor 排序→buzzBeepBlink→notifyPostedLocked，NMS:6292）→binder→SystemUI NotificationListener（binder 线程→主线程）→GroupCoalescer→NotifCollection.onNotificationPosted（:393 记账+重建）→onEntryAdded→HeadsUpController（HeadsUpController.java:86）→shouldHeadsUp 判定链→HeadsUpViewBinder 异步绑横幅视图→HeadsUpManager.showNotification（AlertingNotificationManager.java:68）开始计时。

- **12. NotifCollection 为什么采用"事件记账+整表重建"而不是增量修补列表？**

参考答案要点：每个事件进 mEventQueue 后从 mNotificationSet 全量重建产出不可变快照（ShadeListBuilder.buildList:384）——状态无中间态、协调器（气泡等）改写自动参与下次重建、调

试可整表 dump；去掉队列改同步增量会让分组/动画状态机组合爆炸（13 重写管线的根因）。

- **13. onListenerConnected 里 getActiveNotifications 与 getCurrentRanking 存在什么竞态？SystemUI 如何兜底?**

  参考答案要点：两次独立 IPC，中间新通知可能只出现在前者——RankingMap 缺条目；兜底是 getRankingOrTemporaryStandIn（NotificationListener.java:258）填临时占位 Ranking 防下游崩溃（b/146011844），随后真实排名更新会覆盖。

- **14. 若去掉 RankingMap 的 500ms 合帧（MAX_RANKING_DELAY_MILLIS）会出现什么用户可见症状?**

  参考答案要点：连续进度型通知（下载/音乐）每帧触发一次整表重建与重排，通知列表抖动、视觉稳定性差；现在 dispatchRankingUpdate（:189-217）排空队列或超 500ms 才派发，等价于通知排序的防抖。

- **15. 通知更新（同 key 再 notify）时 entry 与 row 如何复用？为什么"不换 entry 对象"是动画不闪的前提?**

  参考答案要点：NotifCollection.postNotification 命中 else 分支：撤销暂挂状态后仅 setSbn+记账 EntryUpdatedEvent（NotifCollection.java:473-487）；rowExists 时 inflateViews 只 reset+重绑内容（NotificationRowBinderImpl.java:135-140）。entry/row 引用不变，ExpandableNotificationRow 的位移动画与 expand 状态才能跨更新保留。

- **16. 锁屏上某条通知"显示/隐藏"由谁决定？NMS 与 SystemUI 各管哪一段?**

  参考答案要点：NMS 管 visibility 字段与锁屏用户开关数据（书2 侧），SystemUI 侧由 KeyguardNotificationVisibilityProvider.shouldHideNotification 收口；该检查也进了 canAlertCommon（NotificationInterruptStateProviderImpl.java:291 起）——锁屏不让看的通知连横幅都不弹。

- **17. 本册机制与 Binder 十跳（书3）的交汇点至少举三处。**

  参考答案要点：startServiceAsUser（SystemServer:3188，第六章启动）；mBarService.registerStatusBar(mCommandQueue)（CentralSurfacesImpl.java:951，SystemUI 把下行通道端点交给 NMS）；notifyPostedLocked→Listener（上行事件）；setNotificationsShown 回写（HeadsUpController.java:138）；unsnoozeNotification（NotificationListener.java:239）。

第三级：设计题

- **18. 若要求"横幅最短显示时间按渠道重要度自适应"，你会在哪里改？如何不回归已有 snooze 逻辑?**

  参考答案要点：只动 AlertingNotificationManager.updateEntry（:318-334）的 mMinimumDisplayTime 赋值源（改为按 entry.getImportance() 查表），保留 calculateFinishTime 与 isSticky 分支；用现有 HeadsUpManager 单元测试（真实源码附 dagger 测试套件）回归 snooze/延寿路径，判定（shouldHeadsUp）一行不动。

- **19. 设计"SystemUI 启动耗时回归检测"方案：用什么 trace 标签、比较什么基线、如何避免机型差异误报?**

参考答案要点：SystemUIApplication:244 已写 SystemUIBootTiming 的 TimingsTraceLog——perfetto 抓取后按 StartServices* 逐组件切片；基线取同机型连续 N 次开机中位数，按类名字典序逐项对齐（顺序确定性保证可比）；超阈值（如 p95 的 1.5 倍）报警，机型差异用相对占比而非绝对值。

- **20. 假设把通知阴影窗口拆成"锁屏窗口"+"通知列表窗口"两个窗口，列出至少三个必须重新解决的问题。**

参考答案要点：其一，触摸仲裁——锁屏上滑到列表要跨窗口移交（WMS 级 touch region 协商）；其二，Insets 原子性——两套 fitInsets 策略在挖孔/手势区出现接缝；其三，模糊与背景——blur 按窗口算，跨窗口的半透明背景层归属要重设计；再加：窗口显隐时序（第三方的 WMS 动画两段化）、token 归组与层级竞态。

术语表

（按术语拼音排序；源码锚点以 Android 13 为准。）

术语	解释
崩溃隔离	SystemUI 独立进程的收益：UI 崩溃不影响 system_server，init 检测 critical 自动重启
Binder 令牌	attach() 里 mLp.token = new Binder()，WMS 用它做窗口归属与权限校验
合批器	GroupCoalescer，挂在 NotificationListener 直下，把同组通知的连发事件合并后再进 NotifCollection
横幅通知（HUN）	高重要度通知在屏幕顶部的临时横幅形态；判定链入口 HeadsUpController.onEntryAdded
快照	TaskSnapshotController 在 Activity 切后台时经 SurfaceFlinger 截图，最近任务"秒开"的视觉载体
排名表	RankingMap，NMS 随每次增删/排名变化下发的全量排名数据，SystemUI 各层以此对齐
排序提取器	NotificationSignalExtractor 链，逐个调整通知排名分（重要度/勿扰/分组…），可插拔
气泡	Bubbles，对话类通知的浮动形态；与 HUN 在 shouldHeadsUpWhenAwake 中互斥
启动总线	CoreStartable 微服务模型：onStart/onBootCompleted/dump 三件套，经 @IntoMap 收集、TreeMap 排序启动
生命周期扩展器	NotifLifetimeExtender，可否决通知删除（等动画/等交互结束），NotifCollection 的否决机制之一

术语	解释
锁屏可见性	锁屏上通知显示与否的规则；SystemUI 侧收口在 KeyguardNotificationVisibilityProvider
特权应用	SystemUI 的身份：uid=SYSTEM、platform 签名、独立进程；可注册全量通知监听
图标缓存	LauncherModel 的 IconCache，App 图标/标签跨启动持久化缓存
通知行	ExpandableNotificationRow，单条通知的根视图；row 骨架+内容视图三小段异步构建
通知监听器	NotificationListener，SystemUI 注册的特权 NotificationListenerService，通知入 SystemUI 的唯一入口
通知集合	NotifCollection，NMS 事件→NotificationEntry 集合的账本层；mEventQueue 记账+整表重建
通知通道	NotificationChannel，8.0 起发布通知的前提；"通知偏好组"载体，NMS 三道闸第一道
新通知管线	NotifPipeline，13 已默认装配的重建式架构：事件→快照→渲染，取代增量修补的旧管线
挖孔适配	layoutInDisplayCutoutMode=ALWAYS，通知阴影窗口内容延伸进刘海区的设置
勿扰模式	ZenMode，NMS 提取器与 SystemUI 侧 shouldSuppressPeek 双层生效的静默策略
息屏呼吸	AOD pulse，dozing 态下 HUN 的替代形态，判定走 shouldHeadsUpWhenDozing
影子副本	mLpChanged，通知阴影窗口参数的本地副本，改后统一 apply 到 WMS，避免频繁 IPC
整表重建	NotifCollection 收到事件后从 mNotificationSet 全量重过滤/分组/排序，产出不可变列表快照

排障手册：SystemUI 与通知问题定位

1. dumpsys 定位状态 → 2. logcat 查看日志 → 3. 深入各章。

深度实战：SystemUI 与通知的排障场景分析

场景一：幽灵通知——点击/清除后通知行不消失

症状：用户点掉或左滑删除某条通知，图标/通知行仍显示；或 App 已 cancel 而面板依旧。

定位命令：

text 幽灵通知定位命令（可直接执行）

```
adb shell dumpsys notification --noredact | grep -B2 -A15 "com.example.app"
adb shell dumpsys activity service SystemUI | grep -B2 -A10 "NotifCollection"
adb logcat -s NotificationService:D NotificationListener:D
```

根因：三条可能链路。其一，NMS 侧 record 未删（App cancel 走的 key 与发布不一致，tag/id 不匹配）；其二，事件已下发但 NotifCollection 的生命周期扩展器（NotifLifetimeExtender）持续否决删除——动画未结束/交互未完成时 entry 留在集合里；其三，NMS→SystemUI 的 binder 回调被 SELinux 或 listener 解绑吞掉（avc denied）。

修复：核对 dumpsys 里 record 的 key 与 cancel 参数；在 NotifCollection dump 分节查 mFutureDismissals 是否堆积；SELinux 路径用 `adb logcat | grep avc` 取证后补 sepolicy；应用侧用同一 tag/id 组合重发取消。极端情况 force-stop 该 App，NMS 将整包清理。

场景二：通知不响铃、不弹横幅

症状：同一通知在有的设备响/横幅，有的设备完全静默只进阴影。

定位命令：

text 横幅/响铃判定链排查命令

```
adb shell dumpsys notification --noredact | grep -B3 -A20 "com.example.app"
adb shell settings get global zen_mode
adb shell dumpsys notification_manager | grep -A3 mZenMode
adb shell settings put global heads_up_notifications_enabled 1
```

根因：按第九章判定链逐闸排查——渠道重要度低于 IMPORTANCE_HIGH（dumpsys 看 importance 字段）；勿扰（zen_mode 非 0 或 DND 规则过滤 suppressPeek）；用户对包做过横幅 snooze（policy/HeadsUpManager 的 mSnoozedPackages）；锁屏可见性规则在 canAlertCommon 否决；或 NMS 侧 buzzBeepBlink 已静默（第二章三道闸的通道声音配置）。

修复：App 侧确保渠道 importance>=HIGH 且带声音；用户侧检查勿扰规则与该应用的横幅开关；系统侧确认 heads_up_notifications_enabled 未被调试时关掉。

场景三：SystemUI 反复重启——状态栏闪黑

症状：开机后状态栏周期性消失重画，logcat 出现成组 AndroidRuntime 崩溃。

定位命令：

text SystemUI 循环崩溃取证命令

```
adb logcat -b crash | grep -A30 "com.android.systemui"
adb logcat -s AndroidRuntime:E SystemUIService:E
adb shell dumpsys activity processes com.android.systemui | grep -i -E "pid|crash"
```

根因：SystemUI 崩溃→init 按 critical 重启→再次崩溃形成循环。常见三类：Dagger 注入图缺绑定（OEM 定制漏绑 CoreStartable，编译期在 AOSP 上能过、运行期 OEM 分支才炸）；主线程死锁/超时触发 watchdog；插件（PluginManager 加载的第三方 plugin）抛异常。开机早期崩溃还会叠加第六章的 boot 竞态——onBootCompleted 双保险没接住时部分组件停在半初始化态。

修复：按 crash 堆栈首帧定位模块；对缺绑定用 `adb shell dumpsys activity service SystemUI` 看 CoreStartable 清单缺了谁；插件崩溃先 `adb shell setprop persist.sysui.plugin` 指向空目录排除；watchdog 类用 ANR trace（`/data/anr/`）查主线程持有者。

场景四：下拉阴影卡顿或无响应

症状：下拉手势跟手性差、通知列表滚动掉帧，甚至面板收不回去。

定位命令：

text 下拉阴影卡顿定位命令

```
adb shell dumpsys window NotificationShade
adb shell dumpsys gfxinfo com.android.systemui framestats
adb shell perfetto -o /data/misc/perfetto-traces/trace.pftrace -t 10s sched gfx view am wm
```

根因：三类。其一，窗口参数被高频 apply——dumpsys window NotificationShade 看属性变更频率（第七章影子副本机制被绕过，模块直接改 mLp）；其二，NotifPipeline 重建风暴——每帧触发的排名更新没被 500ms 合帧拦住（第八章 dispatchRankingUpdate），或某协调器在 onAfterRenderList 里做重活；其三，row 内容 rebind 泄漏到主线程（第九章三小段中某段退化为同步）。

修复：让窗口参数改动收口到 NotificationShadeWindowController 单点；排查高频 onNotificationRankingUpdate 来源（进度条类通知建议改 progress 增量）；perfetto 里对比 NotifPipeline 重建切片与 Vsync 周期，重建超 8ms 即优化过滤链。

场景五：锁屏不显示任何通知

症状：解锁后通知都在，锁屏面板为空。

定位命令：

text 锁屏通知排查命令

```
adb shell settings get secure lock_screen_show_notifications
adb shell dumpsys notification --noredact | grep -A8 visibility
adb shell dumpsys activity service SystemUI | grep -B3 -A8 "KeyguardNotification"
```

根因：settings secure 锁屏通知总开关为 0；或通知 visibility 为 SECRET/-1 且锁屏未设信任（NMS 侧已标隐藏，书2 数据流）；或 KeyguardNotificationVisibilityProvider 的动态隐私规则（DynamicPrivacyController）判定工作资料/多用户场景隐藏。

修复：用户侧开锁屏通知开关；App 侧发布时设 VISIBILITY_PUBLIC 并提供公开版样式；系统侧核对 KeyguardViewMediator 显示态与 NotificationShadeWindowController.setKeyguardShowing 的同步（第七章 attach 注释⑩的竞态反向场景）。

SystemUI 核心速查表

机制	一句话	源码锚点	所在册
特权 App	SystemUIApplication:62 extends Application	SystemUI	本册
CoreStartable	微服务注册制（每个组件独立启停）	CoreStartable.java:34	本册
通知通道	8+ 必须先 createNotificationChannel	NotificationChannel	本册
NMS 排序	SignalExtractor 链（Importance/ZenMode/...）	NMS	本册
速率限制	频次过高→mRateLimitExceeded→丢弃	NMS:6397	本册
Bubbles	通知的浮动形态（对话类）	BubbleController	本册
TaskSnapshot	Activity 切后台时 SF 截图	TaskSnapshotController	本册
Launcher3	StatefulActivity<LauncherState> 状态驱动	Launcher.java:238	本册
IconCache	App 图标/标签跨启动持久化	LauncherModel:87	本册